Python for Energy System Modelling: Facilitator Pack

This guide sets out how the one-hour workshop runs: the session plan, preparation, the group results form, the expected answers and the debrief. Students work in small groups through two modelling tasks in a Jupyter notebook (JupyterLab through Anaconda on university computers, or Google Colab on personal laptops), record their predictions and results in a shared form, and finish with a short quiz and feedback form.

Session plan

Purpose. The workshop introduces **energy system modelling and analysis** with Python. It is not a programming course: the model code is provided, and students run it, interpret the results, test scenarios and explain what drives them. They complete only three short lines of code, with hints.

Audience: Postgraduate students in energy and sustainable engineering, possibly with some undergraduates. No programming experience is assumed.

Time	Activity	Materials
00 to 15	Introduction: energy system modelling, Python and PyPSA; forming groups and assigning roles	Slides; student guide and GitHub QR codes
15 to 20	Set-up: open the notebook and run sections 1 to 3	Notebook sections 1 to 3
20 to 35	Task 1: a home with solar and a battery	Notebook section 4; results form questions 1 to 4
35 to 50	Task 2: the merit order and a carbon price	Notebook section 5; results form questions 5 to 10
50 to 60	Debrief comparing the groups' results (5 minutes), then quiz and feedback (5 minutes)	Live form summary; quiz QR code

Learning outcomes. By the end of the session, students should be able to:

1. Load and check a year of hourly energy data in Python;
2. Build a simple hour-by-hour battery model and calculate the self-consumption rate, self-sufficiency rate, grid imports and net energy cost;
3. Explain how battery losses and power limits change the result, and why an idealised model gives an upper bound;
4. Build and solve a small least-cost dispatch model in PyPSA;
5. Explain how a carbon price changes the merit order, the clearing price and emissions, and state the main limitations of each model.

Group roles. Each group of three or four splits the work across three screens and agrees one answer before it is entered.

Role	Screen	Responsibility

Role	Screen	Responsibility
Modeller	Notebook	Runs the cells and completes the code
Reviewer	Student guide and learning resources	Reads each step aloud, explains key terms, checks the method against the resources
Reporter	Group results form	Verifies numbers and units, records the group's agreed predictions and results
Second reviewer (groups of four)	Any	Supports the Reviewer and leads the discussion

Roles swap between the tasks, so every student takes a turn as Modeller.

Materials. Student notebook and instructor notebook (.ipynb files, run in JupyterLab through Anaconda or in Google Colab); student guide with key terms and learning resources; group results form (Google Forms, 10 questions); quiz and feedback form; slides. The GitHub repository has two folders. Participants receive the link to the student folder only (student notebook, data, student guide and slides). The instructor notebook, the form script and this pack are in the instructor folder.

Before the session

Pre-session message for students, to be circulated two or three days before. Fill in the date and the links.

> ***Python for energy modelling workshop***
>
> *We will work in JupyterLab through Anaconda Navigator, which is available on university computers through the university's software portal. No Python experience is needed, and you will work in groups of three or four.*
>
> *If you would like to prepare (optional, about five minutes): on any university computer, open Anaconda Navigator, launch JupyterLab, open the notebook from the link below and run the first two code cells. This installs the modelling software in your account, so it is ready on the day. If anything fails, bring the error message and we will fix it at the start.*
>
> *If you prefer your own laptop, you can open the same notebook in Google Colab (colab.research.google.com, then File > Upload notebook).*

Logistics to confirm with the host: the expected number of students; that the room's computers can reach the internet from JupyterLab (the first code cell installs PyPSA with pip and the notebook downloads solar data from PVGIS); how students will receive the notebook file (a shared folder, the module page or a download link); and whether a demonstrator is available to support groups during the tasks.

Test run. Before the day, run the instructor notebook from start to finish once on a university computer of the type participants will use, through Anaconda Navigator. The first run installs PyPSA, which takes

one or two minutes; later runs skip this step. The install cell detects the pandas version that Anaconda provides and installs the matching tested PyPSA release, without changing any existing library.

Running the notebook: JupyterLab and Colab

Students have step-by-step instructions and a button table in the student guide ("Getting started"). In short:

- **University computers:** the university's software portal, then Anaconda Navigator, then **Launch** on the JupyterLab tile. Upload the notebook to Documents or the H: drive in the File Browser, open it, and run the first cell with Shift + Enter.
- **Personal laptops:** Google Colab, **File > Upload notebook**, then run the first cell.
- **Run everything:** Run > Run All Cells (JupyterLab) or Runtime > Run all (Colab). In the student notebook this stops at the battery blanks in section 4.4 until they are filled in, which is expected.

Issues identified during testing, and the recommended response to each:

What students see	What to do
A kernel list on opening, such as an Anaconda option with py39	Choose **Python 3 (ipykernel)** or the newest option with Python 3.11 or later. The first cell stops with a clear message on older Python
"Note: you may need to restart the kernel" after the install	Ignore it; the cell continues and prints "PyPSA … is ready."
"PyPSA is not available yet"	Kernel > Restart Kernel (JupyterLab) or Runtime > Restart session (Colab), then run the first cell again
Colab warns that the notebook was not authored by Google	Click **Run anyway**
"PVGIS could not be reached"	Check the internet connection and run the cell again; the data is saved after the first successful download
A cell shows **[*]** for a long time	It is still running. The first run takes one or two minutes for the install; later cells take seconds
Where are the results?	In the **outputs** folder next to the notebook; the last cell collects everything in workshop_outputs.zip

Nothing to print. Scenarios are set by group number in the notebook, and the common-errors guide sits in a collapsible box at the start of Task 1. The slides carry three QR codes: the student guide, the group results form, and the quiz and feedback form.

Groups, scenarios and timing

Assign each group a number at the start and announce it clearly. Groups type it as `MY_GROUP` in section 4.8, and the notebook picks both scenarios from this table. With five groups, groups 1 to 5 cover every Task 2 scenario. Groups 9 and above wrap round to group 1.

Group	Task 1 scenario	Task 2 scenario
1	A, 6 kWp of solar	P, carbon price 40
2	B, 2.5 kWp of solar	Q, carbon price 120
3	C, 3,800 kWh demand	R, gas at 140 GBP/MWh
4	D, 1,600 kWh demand	S, 30 GW of wind
5	E, 10 kWh battery	T, calm day
6	F, older battery	R, gas at 140 GBP/MWh
7	G, export at 4 p/kWh	T, calm day
8	B, 2.5 kWp of solar	S, 30 GW of wind

Task 1 (15 minutes, after the 5-minute set-up): 10 minutes for the battery rule, the four-hour test and sections 4.5 to 4.7, then 5 minutes for the group scenario in section 4.8. At the 10-minute mark, groups still working on the blanks should open hint 2 and continue. Everything from section 4.6 onwards runs without the blanks: the group clicks in the first code cell of 4.6 and uses **Run > Run Selected Cell and All Below** (JupyterLab), or runs that cell and then **Runtime > Run after** (Colab).

Task 2 (15 minutes): 10 minutes for the prediction, the runs at carbon prices of 0 and 80, and Figures 6 and 7, then 5 minutes for the group scenario in section 5.6. The stretch tasks are intended only for groups that finish early.

Google Form for group results

Run the script **Create_Workshop_Results_Form.gs** once: go to script.google.com, create a new project, paste the script, run `createWorkshopForm` and allow access. The log gives you the student link (for the QR code), your edit link and a linked results sheet. Open the edit link once and click **Publish** if Google shows that button.

It is a plain data-collection form, not a graded quiz: one response per group, entered by the Reporter as the group works. Only the group number is required, and response editing is on. During the debrief, show the **Responses, Summary** view on the projector; it fills in as groups submit. If the script is run again, it creates a new form and sheet, so delete any older copies from Drive.

No.	When	Question
1	Start	Group number (required)
2	Before 4.5	Prediction: self-sufficiency rate with a 5 kWh battery (%)

No.	When	Question
3	4.8	Scenario self-sufficiency rate with the battery (%)
4	4.8	Scenario annual saving from the battery (GBP/yr)
5	Before 5.4	Prediction grid: at 80 GBP/tCO2, do the peak price and direct CO2 go up, stay the same or go down?
6	5.4	Peak-hour price at 80 (GBP/MWh)
7	5.4	Direct CO2 emissions at 80 (tCO2)
8	5.6	Scenario peak-hour price (GBP/MWh)
9	5.6	Scenario direct CO2 emissions (tCO2)
10	End	One sentence explaining one scenario result

Expected values for questions 6 and 7: 89.6 GBP/MWh and 139,342 tCO2 (against 60.0 and 253,504 at a carbon price of 0).

Predictions for the scenarios are agreed aloud within each group, not entered in the form. Say “self-sufficiency rate” with its meaning the first time: the share of the home's yearly demand met by its own solar, directly or through the battery.

Answer key

Task 1 values below come from the real PVGIS-SARAH3 data for 2019 (986 kWh/kWp), run in Anaconda on 17 October 2026, and match the **instructor-only** answer-key cells in the instructor notebook. Reference home with the realistic battery: self-sufficiency rate 78.6 %, annual saving from the battery 129.0 GBP/yr (no battery 38.9 %; idealised battery 79.9 %).

Scenario	Self-sufficiency rate with battery (%)	Annual saving from the battery (GBP/yr)	Point to draw out
A, 6 kWp	83.7 (+5.1)	136.8 (+7.8)	More solar raises self-sufficiency but lowers self-consumption: more is exported
B, 2.5 kWp	69.4 (−9.2)	111.2 (−17.8)	Less surplus to store, so the battery sits partly empty
C, 3,800 kWh	65.9 (−12.7)	149.2 (+20.2)	The battery empties every evening, so it is used fully; more demand is left for the grid
D, 1,600 kWh	88.7 (+10.1)	97.5 (−31.5)	Little evening demand, so much of the stored energy has nowhere useful to go
E, 10 kWh	83.8 (+5.2)	145.9	Diminishing returns: doubling the battery adds

Scenario	Self-sufficiency rate with battery (%)	Annual saving from the battery (GBP/yr)	Point to draw out
		(+16.9)	only about 5 points
F, older battery	77.8 (−0.8)	118.9 (−10.1)	Losses and the power limit cost little in this home
G, 4 p export	78.6 (no change)	217.2 (+88.2)	Energy flows are identical; the battery is worth more when exporting pays less

Task 2 does not depend on the solar data, so these are the exact values.

Scenario	Peak-hour price (GBP/MWh)	Set by	Direct CO2 (tCO2)	Consumer payments (GBP m)	Point to draw out
Reference, 80	89.6	Gas	139,342	77.9	Coal is pushed out by the carbon price
P, 40	74.8	Gas	253,504	64.2	Below the 47 GBP/t switch point, coal runs again: cheaper power, 82 % more CO2
Q, 120	104.4	Gas	139,342	90.7	Price rises but emissions do not fall further: coal is already out
R, gas 140	169.6	Gas	253,504	134.9	Expensive gas brings coal back despite the carbon price; the switch point rises to about 198 GBP/t
S, 30 GW wind	89.6	Gas	86,691	66.9	Same peak price, but 38 % less CO2 and lower payments: wind displaces gas off-peak
T, calm day	107.0	Coal	208,749	86.4	Without enough wind, coal is needed at the peak and sets the price

If a group asks about closing coal entirely: at a carbon price of 80, the result is identical to the reference, because the carbon price has already pushed coal out of the merit order.

If a group asks why Figure 7 shows the price dropping at 23:00 when coal runs ahead of gas (carbon price 0, and scenarios P and R): at that hour wind, nuclear and coal together exactly equal demand, so the price is not unique and the solver reports the lower value. This is the degeneracy explained in section 5.3, not an error.

Debrief slide

Replace or add to slide 12 with two compact tables, filled live from the form's responses. Title: **What changed, and why?**

Task 1 scenario	Predicted	Self-sufficiency rate	Saving (GBP/yr)
Reference home		78.6 %	129.0
A to G, one row per group	up / down		

Task 2 scenario	Predicted price	Peak price (GBP/MWh)	CO2 (t)
Reference, 80		89.6	139,342
P to T, one row per group	up / same / down		

Talking points, in this order:

1. Ask which group's prediction was most wrong, and why. Unexpected results are often the most instructive.
2. Compare A with D: both raise self-sufficiency, but for opposite reasons (more supply against less demand).
3. Compare E with G: doubling the battery adds only about 5 points, but a lower export price makes the same battery worth much more. The value of storage depends on prices, not only on kWh.
4. Compare Q with R and T: a carbon price changes emissions only when it changes the merit order, and fuel prices and the weather can undo it.
5. Close on the limits: these are teaching models with one weather year, a smooth demand profile and one stylised day.